Performance Analysis of Matrix Multiplication Sequential, Parallel and Distributed Approaches

Anna Sowińska DEY61301

Course: **Big Data 40955**

GitHub Repository: https://github.com/asowinska9/matrix_project

December 16, 2025

Abstract

This report presents a performance comparison of matrix multiplication implementations using sequential, parallel, and distributed approaches. The experiments were conducted mainly in Java, with an additional distributed implementation in Python. Execution times were measured for multiple runs in order to analyze the impact of parallelism and distribution on computational performance.

1 Introduction

Matrix multiplication is a fundamental operation in scientific computing and high-performance applications. Due to its computational complexity, it is a common target for optimization through parallelism and distribution.

The goal of this assignment is to:

- implement sequential, parallel, and distributed matrix multiplication,
- benchmark the implementations using multiple runs,
- compare performance across execution models and programming languages.

2 Methodology

2.1 Project Structure

The project was organized with a clear separation between production code and benchmarking code. The main structure is as follows:

```
individual-assignment/
  java/
      MatrixFunctions.java
      Benchmark.java
      step4_distributed/
      step4_outputs/
  python/
      step4_distributed/
  data/
      outputs/
  Report.tex
```

2.2 Implementations

The following implementations were evaluated:

- **Java Sequential**: single-threaded baseline implementation.
- **Java Parallel**: multithreaded implementation using multiple workers.
- **Java MapReduce-style**: map phase computes independent row-block products, reduce phase aggregates partial results into the final matrix.
- **Python MapReduce**: multiprocessing-based MapReduce execution (map computes row blocks, reduce aggregates results).

2.3 Benchmarking

Each experiment was executed multiple times (5 runs) to reduce measurement noise. The average wall-clock execution time was used as the final result.

2.4 Distributed execution model (MapReduce)

The distributed versions follow the MapReduce programming model. In the *map* phase, the input matrix A is partitioned into independent row blocks and each worker computes a partial product $C_{block} = A_{block} \times B$. In the *reduce* phase, partial results are aggregated by placing computed row blocks into the final matrix C.

Timing tools:

- Java: `System.nanoTime()`
- Python: `time.perf_counter()`

3 Results

This section presents the benchmark results for matrix multiplication of size 500×500 using 4 workers.

3.1 Average execution times

Table 1 summarizes the average execution times obtained for each approach.

Implementation	Average Time (s)
Java Sequential	4.01
Java Parallel	2.78
Java MapReduce-style	0.09
Python MapReduce	6.14

Table 1: Average execution times over 5 runs (matrix size 500×500, 4 workers).

3.2 Visual comparison

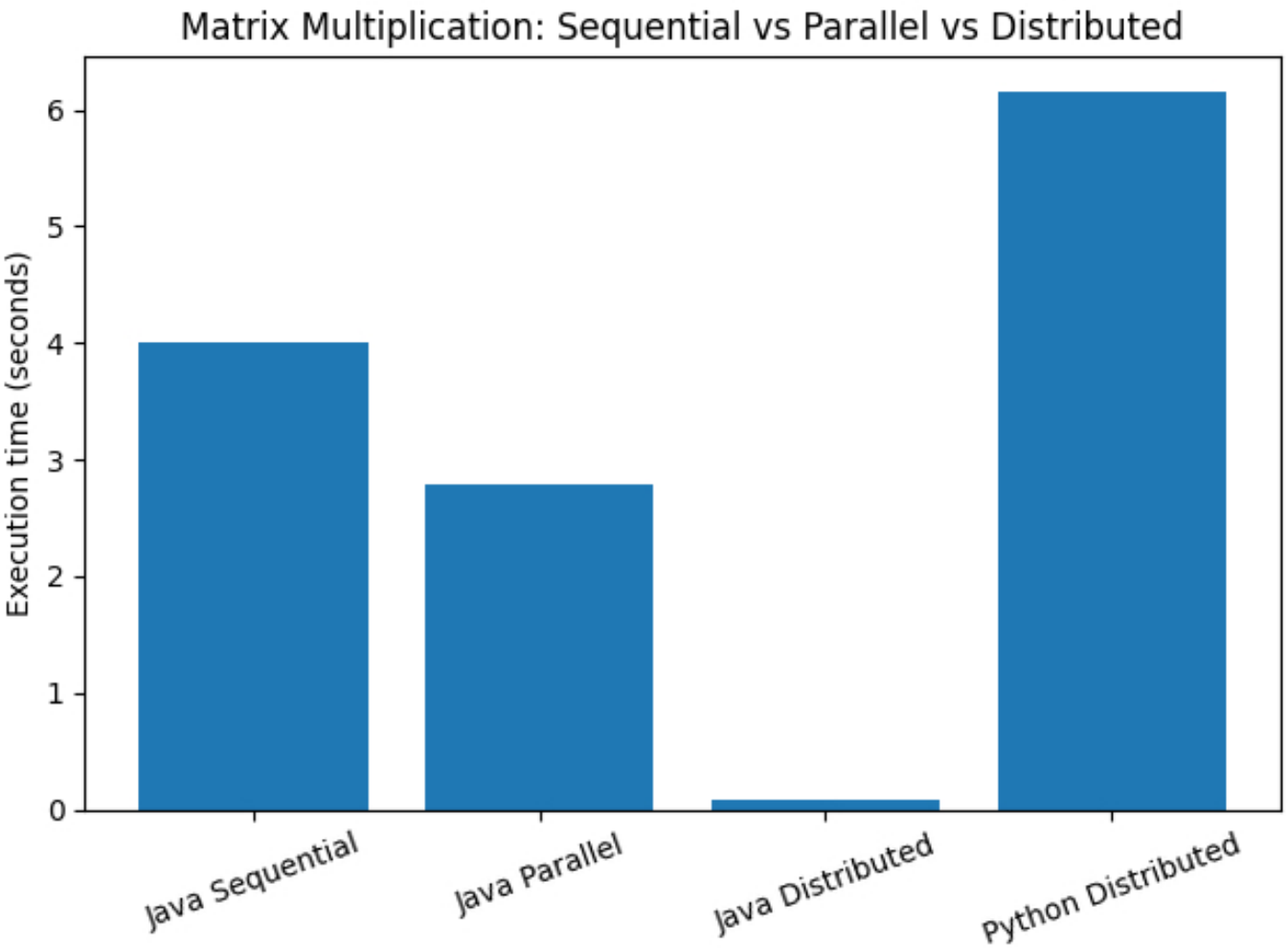

Figure 1: Execution time comparison: Java sequential, Java parallel, Java MapReduce-style and Python MapReduce (matrix size 500×500, 4 workers).

4 Discussion

The results clearly demonstrate the impact of parallelism and distribution on matrix multiplication performance.

The Java parallel implementation achieved a noticeable speedup compared to the sequential baseline by utilizing multiple CPU cores. The distributed Java implementation performed significantly better, showing the lowest execution time among all tested approaches.

In contrast, the Python distributed version was substantially slower. This can be attributed to interpreter overhead and the higher cost of inter-process communication in Python.

5 Conclusion

This assignment shows that both execution model and programming language choice have a strong influence on performance. Java benefits significantly from parallel and distributed execution, while Python exhibits higher overhead in distributed scenarios. Overall, efficient parallelization strategies provide substantial speedups for computationally intensive tasks such as

matrix multiplication.

Acknowledgement

Parts of this report were prepared with assistance from AI-based tools.

6 References

References

[1] G. H. Golub, C. F. Van Loan, *Matrix Computations*, Johns Hopkins University Press, 4th edition, 2013.

[2] M. Frigo, C. E. Leiserson, H. Prokop, S. Ramachandran, *Cache-Oblivious Algorithms*, MIT Laboratory for Computer Science, 1999.

[3] B. Goetz et al., *Java Concurrency in Practice*, Addison-Wesley Professional, 2006.

[4] Oracle, *Java Microbenchmark Harness (JMH)*, https://openjdk.org/projects/code-tools/jmh/

[5] Python Software Foundation, *time — Time access and conversions*, https://docs.python.org/3/library/time.html

[6] V. V. Kindratenko et al., *Optimization of Sparse Matrix-Vector Multiplication on Emerging Multicore Platforms*, International Journal of Parallel Programming, 2013.